

OFFICIAL ENGINEERING & OPERATIONS DOCUMENTATION

INNOVISION — BUG HUNT

Production System Architecture, Database Design, Anti-Cheat Engine & Coordinator Operations Guide

A real-time, browser-based competitive programming and code debugging platform engineered for high-concurrency collegiate hackathons. Featuring automated test execution, multi-round progression state machines, client surveillance anti-cheat guards, and real-time coordinator command centers.

ORGANIZING DEPARTMENT

Artificial Intelligence and Data Science

EVENT ASSOCIATION

INNOVISION Technical Symposium
(2026)

DOCUMENT VERSION & DATE

Version 2.0 (Post Security Audit) •
October 2026

ARCHITECTURE CORE

Next.js 16 • React 19 • Supabase
PostgreSQL • Monaco

0 Table of Contents

1. Executive Summary & Competition Framework	Section 1
2. Complete System Architecture & Technology Stack	Section 2
3. Database Schema, Data Models & Row-Level Security (RLS)	Section 3
4. Student Experience & Competition Arena Engine	Section 4
5. Anti-Cheat Surveillance & Integrity Guard	Section 5
6. Administrator Command Center & Round State Machine	Section 6
7. Complete API Route & Endpoint Specification	Section 7
8. Event Day Operations, Deployment & Emergency Checklist	Section 8

Document Purpose & Confidentiality

This document serves as the single source of truth for software engineers, event administrators, lab technicians, and faculty coordinators managing the **INNOVISION — BUG HUNT** platform. It contains database schemas, API contracts, security configurations, and disaster recovery procedures.

1 Executive Summary & Competition Framework

1.1 Vision & Objectives

INNOVISION — BUG HUNT is a high-octane coding contest hosted by the Department of Artificial Intelligence and Data Science. Unlike conventional competitive programming contests where participants write code from scratch, *BUG HUNT* tests code comprehension, static analysis, logical error identification, and precision debugging under strict time constraints.

1.2 Multi-Round Competition Structure

The competition is structured into three progressive rounds totaling 100 marks. Every registered student begins in Round 1 and can qualify forward according to coordinator criteria:

ROUND	TITLE & FOCUS	DURATION	QUESTIONS	TOTAL MARKS	DEFAULT STATE
Round 1	Easy: Bug Hunt Basics Syntax, arithmetic typos, conditional reversals, loop bounds	10 Minutes	8 Questions	20 Marks	LIVE
Round 2	Moderate: Debugging Challenge Array indexing, dictionary lookups, off-by-one errors, recursion bases	10 Minutes	5 Questions	30 Marks	READY
Round 3	Hard: Final Bug Hunt Algorithmic bugs, sorting, string manipulation, edge-case failure	10 Minutes	3 Questions	50 Marks	FINALISTS
TOTAL COMPETITION METRICS		30 Minutes	16 Questions	100 Marks	—

1.3 Scoring Rules & Deterministic Tie-Breaker Algorithm

In high-stakes hackathons, ties often occur when contestants solve identical questions. The platform evaluates contestants and sorts the real-time leaderboard using an automated 4-tier tie-breaking algorithm:

CRITERION 1 Total Score Aggregate score across all rounds (Max 100). Primary rank order.	CRITERION 2 Round 3 Score Performance on hardest algorithmic challenges (Max 50).	CRITERION 3 Round 2 Score Performance on intermediate debugging challenges (Max 30).	CRITERION 4 Earliest Submission Contestant who submitted solutions earliest in UTC timestamp.
--	---	--	---

2 System Architecture & Technology Stack

2.1 High-Level Architecture Overview

The platform is built as a cloud-native, server-rendered and client-hydrated application leveraging Next.js 16 App Router, TypeScript, and Supabase PostgreSQL.

Architecture Pipeline

```
[ Participant Web Browser ]
├── (1) HTTPS Student Session Cookie (HTTP-Only)
├── (2) Anti-Cheat Surveillance Signals (POST /api/student/anti-cheat)
├── (3) Monaco Code Editor (Browser WASM / WebWorker)
└──
[ Next.js 16 Application Server (Vercel / Node.js) ]
├── Route Handlers & Anti-Cheat Ingestion Engine
├── Automated Test Case Evaluator (Input/Output assertion)
├── Session Management & JWT Token Generator
├── Privileged Supabase Admin Client (SUPABASE_SERVICE_ROLE_KEY)
└──
[ Supabase PostgreSQL 15 Cloud Database ]
├── public.registrations & public.registration_members
├── public.participant_sessions (State & Live Scores)
├── public.submissions (Code snapshots & Test outputs)
├── public.competition_settings & public.rounds
├── public.audit_logs (Tamper-proof compliance log)
└── Row Level Security (RLS) Active Policies
```

2.2 Core Technology Stack Components

Frontend & UI Engine

- **Next.js 16.3.6 (App Router):** High-performance server rendering and streaming.
- **React 19.2.8:** Latest React concurrency features and state primitives.
- **Tailwind CSS 4:** Zero-runtime styling with custom cyber dark-mode palette.
- **Monaco Editor (@monaco-editor/react 4.7):** Industrial-grade VS Code editor inside the browser.
- **Lucide Icons & Canvas Confetti:** Interactive visual feedback and status badges.

Backend & Database Infrastructure

- **Supabase PostgreSQL:** Managed ACID database with connection pooling.
- **Row Level Security (RLS):** Cryptographically enforced table isolation.
- **Service Role Key:** Elevated server-only privileges bypassing public restrictions.
- **Cookie & JWT Authentication:** Tamper-proof session validation for students and coordinators.
- **In-Memory Resilient Fallback:** Zero-downtime offline support if cloud connectivity hiccups.

2.3 Dual-Engine Data Persistence (Hybrid Resilience)

To ensure zero event disruption during high-stress collegiate lab conditions (e.g., intermittent Wi-Fi or firewall blocks), the application employs a **Hybrid Persistence Architecture** (in `src/lib/store.ts`):

- **Primary Engine:** Cloud Supabase database queried via `supabaseAdmin` with transaction integrity.
- **Fallback Engine:** In-memory and static verified seed datasets (`src/data/participants.ts` , `src/data/questions.ts`). If Supabase credentials are unset or the network fails, the system seamlessly handles registrations and test scoring in memory without crashing.

3 Database Schema, Data Models & Row-Level Security

3.1 Relational Schema Architecture

The platform's relational model in Supabase PostgreSQL (defined in `supabase/schema.sql`) consists of 8 core tables:

TABLE NAME	PRIMARY KEY	FOREIGN KEYS / INDEXES	DESCRIPTION & ROLE
registrations	id (UUID)	registration_id (UNIQUE)	Stores approved participant details synced with INNOVISION master database.
registration_members	id (UUID)	registration_id -> registrations	Supports team members and full student profile metadata.
participant_sessions	registration_id (VARCHAR)	registration_id -> registrations	Maintains live state, start/submit timestamps, scores (R1, R2, R3), finalist flag.
submissions	id (UUID)	registration_id , question_id	Stores every code attempt, passed test counts, score, and execution duration.
competition_settings	id (VARCHAR)	Primary: main_settings	Global configuration: tab limits, copy-paste flags, announcements, rules.
rounds	round_number (INT)	Values: 1, 2, 3	Round states (LOCKED, READY, LIVE, PAUSED, COMPLETED), durations, marks.
questions	id (VARCHAR)	round_id -> rounds	Buggy code, hidden/visible test cases, explanations, marks, languages.
audit_logs	id (UUID)	registration_id , event_type	Immutable anti-cheat trail (tab switches, paste attempts, blur events).

3.2 Row-Level Security (RLS) & Threat Mitigation

Threat Model & Vulnerability Resolution (rls_disabled_in_public)

Because client-side web applications expose `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY`, having tables with RLS disabled allows any tech-savvy participant to craft POST/DELETE requests using DevTools to overwrite competitor scores or delete questions.

Enforced Mitigation: RLS is strictly enabled across all tables. The public anonymous role is restricted to reading public settings and round statuses. All mutations, score writes, and participant updates are locked down exclusively to the server-side `service_role` key.

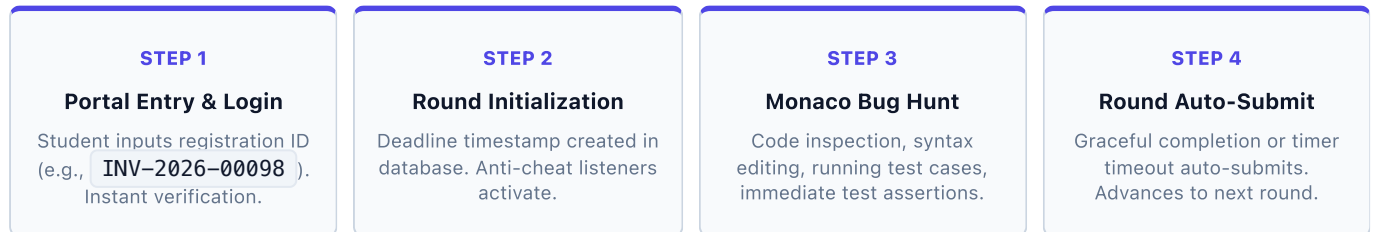
```
-- Production Security Configuration
ALTER TABLE public.registrations ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.registration_members ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.participant_sessions ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.submissions ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.competition_settings ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.rounds ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.questions ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.audit_logs ENABLE ROW LEVEL SECURITY;

-- Anonymous users can strictly read public contest metadata
CREATE POLICY "Public can view settings" ON public.competition_settings FOR SELECT USING (true);
CREATE POLICY "Public can view rounds" ON public.rounds FOR SELECT USING (true);

-- Backend API operations utilize the privileged service role (auth.role() = 'service_role')
```

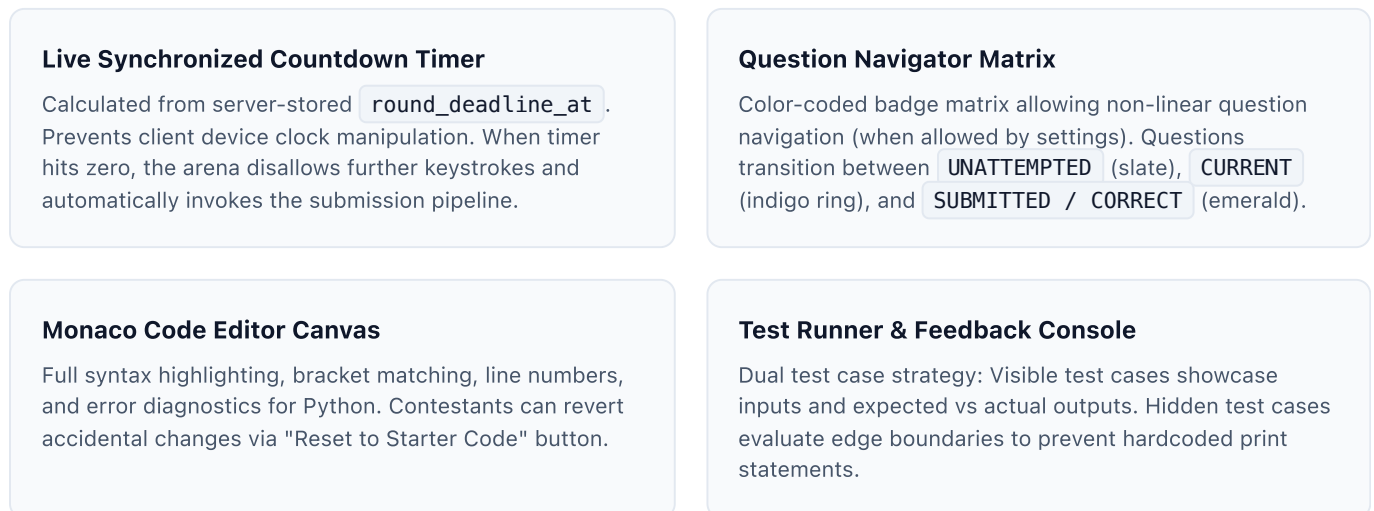
4 Student Experience & Competition Arena Engine

4.1 End-to-End Contestant Journey



4.2 Competition Arena Components

The competition arena (`src/components/CompetitionArena.tsx`) provides a responsive, IDE-like interface tailored for fast debugging:



4.3 Automated Evaluation & Submission Engine

When a student clicks "**Submit Solution**", the payload is transmitted to `/api/student/submit` :

- The server evaluates the submitted code against all test cases associated with the question.
- Score is calculated proportionally: `(passed_cases / total_cases) * question.marks` .
- A permanent snapshot is recorded in `submissions` with timestamp and output logs.
- The student's round total is dynamically recalculated and updated in `participant_sessions` .

5 Anti-Cheat Surveillance & Integrity Guard

5.1 Multi-Vector Integrity Protection

To preserve competitive fairness in computer lab environments, the application embeds an active anti-cheat sentinel (`src/components/AntiCheatGuard.tsx`) running client-side event loops:

SURVEILLANCE VECTOR	DETECTION MECHANISM	ACTION TAKEN	LOGGED EVENT TYPE
Tab / Window Switching	<code>document.visibilitychange</code> & <code>document.hidden</code>	Triggers visual compliance warning; increments violation tally.	TAB_SWITCH
Focus Loss (Window Blur)	<code>window.addEventListener("blur")</code>	Detects split-screen or external messaging applications.	BLUR
Clipboard Paste Attempt	<code>clipboard.addEventListener("paste")</code>	Intercepts paste event (<code>e.preventDefault()</code>); blocks external code insertion.	PASTE_ATTEMPT
Fullscreen Exit	<code>document.fullscreenElement</code>	Alerts student to re-enter fullscreen mode immediately.	FULLSCREEN_EXIT
Developer Tools Opening	Window resize threshold analysis	Prevents inspection of source code and client variables.	DEVT00LS

5.2 Violation Thresholds & Enforcement

Every detected violation fires an asynchronous beacon to `/api/student/anti-cheat` :

- The violation is appended to the immutable `audit_logs` table with student registration ID, timestamp, and context details.
- The participant's `suspicious_count` in `participant_sessions` is incremented.
- If `suspicious_count >= tab_switch_limit` (configured in settings, default: 3), the server can flag the session for coordinator review or set status to `DISQUALIFIED / LOCKED` .
- Coordinators can view real-time anti-cheat alerts directly on the live Admin Dashboard.

Coordinator Discretion & Manual Unlocking

Network fluctuations or accidental desktop notifications can occasionally cause focus blur. The platform therefore provides an **Admin Unlock / Reset** function in the coordinator panel to reinstate accidentally locked students.

6 Administrator Command Center & Operations

6.1 Admin Portal Capabilities

The coordinator interface (accessible at `/admin/dashboard`) provides complete authority over the event lifecycle through 9 specialized command views:

Live Telemetry Dashboard

Real-time counts of logged-in students, active submissions, average scores, and live anti-cheat incident feeds.

Participant Manager

Filter contestants by round, view scores, inspect IP addresses, reset sessions, or flag disqualifications.

Round State Controller

Execute state transitions across Round 1, 2, and 3: `LOCKED`, `READY`, `LIVE`, `PAUSED`, `COMPLETED`.

Finalist Selection Engine

Filter top contestants after Round 2 based on cumulative score cutoffs and promote them to Round 3.

Submissions Inspector

Browse every code submission by question, view exact code diffs, execution times, and test outputs.

Dynamic Leaderboard

Real-time ranked board with tie-breaker calculations. Can be made public or kept hidden until awards.

Question Bank Repository

Browse all 16 competition questions, view buggy code, canonical solutions, and hidden test vectors.

Contest Settings

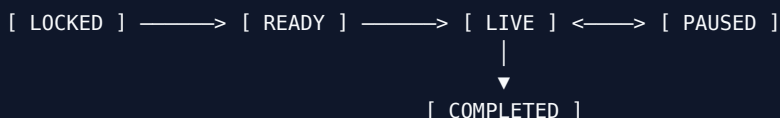
Configure anti-cheat strictness, navigation policies, copy-paste flags, and tie-breaker strings.

Live Announcement Broadcaster

Push emergency or informational broadcast banners to all student competition screens instantly.

6.2 Round State Machine Workflow

Coordinators transition rounds sequentially through the following state lifecycle:



- **LOCKED:** Students cannot access questions or view starter code.
- **READY:** Round is queued. Contestants see a "Get Ready" countdown screen.
- **LIVE:** Countdown clock ticks down. Editor is unlocked and active. Submissions accepted.
- **PAUSED:** Coordinators freeze timers across all stations for announcements or lab issues.
- **COMPLETED:** Round closes. All unsubmitted work auto-evaluates. Finalist selection unlocks.

7 Complete API Route & Endpoint Specification

All endpoints accept and return `application/json`. Authentication is validated via HTTP-only cookies or bearer headers.

7.1 Student API Endpoints

METHOD & PATH	AUTH	REQUEST PAYLOAD	RESPONSE PAYLOAD	FUNCTION
POST <code>/api/student/verify</code>	Public	<code>{ registrationId: string }</code>	<code>{ success: boolean, participant: Participant }</code>	Validates ID, initiates session cookie, and returns student profile.
GET <code>/api/student/round-session</code>	Student Cookie	Query: <code>?round=1</code>	<code>{ activeRound, deadline, questions, status }</code>	Returns active round parameters and questions (solution stripped).
POST <code>/api/student/submit</code>	Student Cookie	<code>{ questionId, code, roundId }</code>	<code>{ score, testsPassed, totalTests, results }</code>	Evaluates code, updates round scores, records submission snapshot.
POST <code>/api/student/anti-cheat</code>	Student Cookie	<code>{ eventType, details }</code>	<code>{ success, suspiciousCount, warning }</code>	Logs compliance events to <code>audit_logs</code> ; increments violation count.
POST <code>/api/student/complete-round</code>	Student Cookie	<code>{ roundNumber: number }</code>	<code>{ success, nextRoundUnlocked, totalScore }</code>	Marks round as completed and handles transition to subsequent round.
POST <code>/api/student/logout</code>	Student Cookie	—	<code>{ success: true }</code>	Clears student session cookies and ends active tracking.

7.2 Administrator API Endpoints

METHOD & PATH	AUTH	PAYLOAD / PARAMETERS	DESCRIPTION
POST /api/admin/login	Public	{ email, password }	Validates admin credentials, issues signed JWT session cookie.
GET /api/admin/dashboard/stats	Admin Cookie	—	Returns total registered, active contestants, average scores, live audit feed.
GET/POST /api/admin/participants	Admin Cookie	Action: reset, unlock, disqualify	Retrieves participant roster or updates participant operational status.
GET/POST /api/admin/rounds	Admin Cookie	{ roundNumber, status, duration }	Updates round state machine (e.g., transitions Round 1 to COMPLETED).
GET/POST /api/admin/finalists	Admin Cookie	{ registrationIds: string[] }	Promotes qualified students to finalist tier for Round 3 access.
GET /api/admin/submissions	Admin Cookie	Query: ?questionId=&round=	Fetches raw submitted code and test outputs across contestants.
GET/POST /api/admin/settings	Admin Cookie	CompetitionSettings object	Reads or updates global rules, tab limits, copy-paste permissions.

8 Event Day Operations, Deployment & Checklist

8.1 Pre-Flight Environment Variables Reference (`.env.local`)

VARIABLE KEY	SCOPE	DESCRIPTION & EXPECTED VALUE
<code>NEXT_PUBLIC_SUPABASE_URL</code>	Public / Client	Project HTTPS URL (e.g., <code>https://yduwzfzdugtdlrkreagr.supabase.co</code>)
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Public / Client	Supabase Anon key (read-restricted by RLS policies).
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Server Only	Privileged Service Role Secret bypassing RLS for backend routes.
<code>ADMIN_EMAIL</code>	Server Only	Master coordinator login email (e.g., <code>dineshh2519@gmail.com</code>).
<code>ADMIN_PASSWORD</code>	Server Only	Encrypted or secure coordinator portal access password.
<code>ADMIN_JWT_SECRET</code>	Server Only	High-entropy cryptographic key used for signing coordinator JWT tokens.
<code>STUDENT_SESSION_SECRET</code>	Server Only	Secret key for encrypting participant session cookies.

8.2 Coordinator Event Day Protocol & Timeline

T-minus 60 Minutes (Lab & Infrastructure Verification)

- Launch the web server using `npm run build && npm run start` or verify cloud deployment URL.
- Login to Coordinator Command Center (`/admin/login`) and ensure dashboard metrics load.
- Verify all lab client PCs can load the home page and resolve DNS.
- Confirm Supabase connection status shows **Connected (Cloud Mode)**.

T-minus 10 Minutes (Contestant Seating & Briefing)

- Direct participants to the login page (`/login`) and instruct them to enter their assigned ID.
- Ensure Round 1 is set to **READY** status.
- Brief students on anti-cheat protocols: *"Tab switches, window blurs, and paste attempts are automatically logged and will disqualify after 3 strikes."*

Event Execution (Rounds 1, 2, and 3)

- **3:00 PM:** Coordinator sets Round 1 to **LIVE**. Arena countdown begins.
- **3:10 PM:** Round 1 closes. Review leaderboard. Transition Round 2 to **LIVE**.
- **3:20 PM:** Round 2 closes. Navigate to `/admin/finalists`. Select top qualifying students for Round 3.
- **3:30 PM:** Round 3 (Finals) set to **LIVE**. Finalists compete for the championship.
- **3:40 PM:** Competition finishes. Leaderboard exported for valedictory ceremony.

8.3 Disaster Recovery FAQ

Q: What if a lab PC crashes or the student accidentally closes their browser?

A: All submissions, scores, and active round deadlines are stored in Supabase PostgreSQL. The student simply re-opens the browser, types their registration ID at `/login`, and their exact session and scores will be restored instantly.

Q: What if a student is locked out due to accidental popups or antivirus notifications?

A: The coordinator can navigate to **Admin > Participants**, search for the student's Registration ID, and click "**Unlock Session**" or "**Reset Strikes**" to immediately re-enable their editor.